take these elements as additional method parameters. Besides these optional elements, each message has, at very least, a level, category, and message. These are the key features of the log, so we'll look at each in turn.

26.2.1 Log level: How important is the log message?

Whenever you create a log using `ILogger`, you must specify the *log level*. This indicates how serious or important the log message is, and it's an important factor when it comes to filtering which logs are written by a provider, as well as finding the important log messages after the fact.

You might create an `Information` level log when a user starts to edit a recipe. This is useful for tracing the application's flow and behavior, but it's not important, because everything is normal. But if an exception is thrown when the user attempts to save the recipe, you might create a `Warning` or `Error` level log.

The log level is typically set by using one of several extension methods on the `ILogger` interface, as shown in listing 26.3. This example creates an `Information` level log when the `View` method executes and a `Warning` level error if the requested recipe isn't found.

Listing 26.3 Specifying the log level using extension methods on `ILogger`

```
private readonly ILogger _log;                                    ❶
public async IActionResult OnGet(int id)
{
    _log.LogInformation(                                          ❷
        "Loading recipe with id {RecipeId}", id);                 ❷

    Recipe = _service.GetRecipeDetail(id);
    if (Recipe is null)
    {
        _log.LogWarning(                                          ❸
```